

Data Mining Practice Based on CRISP-DM for Regression and Classification

Course: Data Mining / Machine Learning Practice **Student:** _____ **ID:** _____ **Date:** _____

This experiment follows the **CRISP-DM** (Cross-Industry Standard Process for Data Mining) six-phase methodology to complete two classic data-mining tasks:

Task	Dataset	Type	Target
Regression	Kaggle House Prices (<code>train.csv</code>)	Predict continuous value	<code>SalePrice</code>
Classification	UCI Chronic Kidney Disease (<code>kidney_disease.csv</code>)	Predict binary class	<code>classification</code> (ckd / notckd)

CRISP-DM phases covered: Business Understanding → Data Understanding → Data Preprocessing → Modeling → Evaluation → Result Analysis.

CRISP-DM Workflow

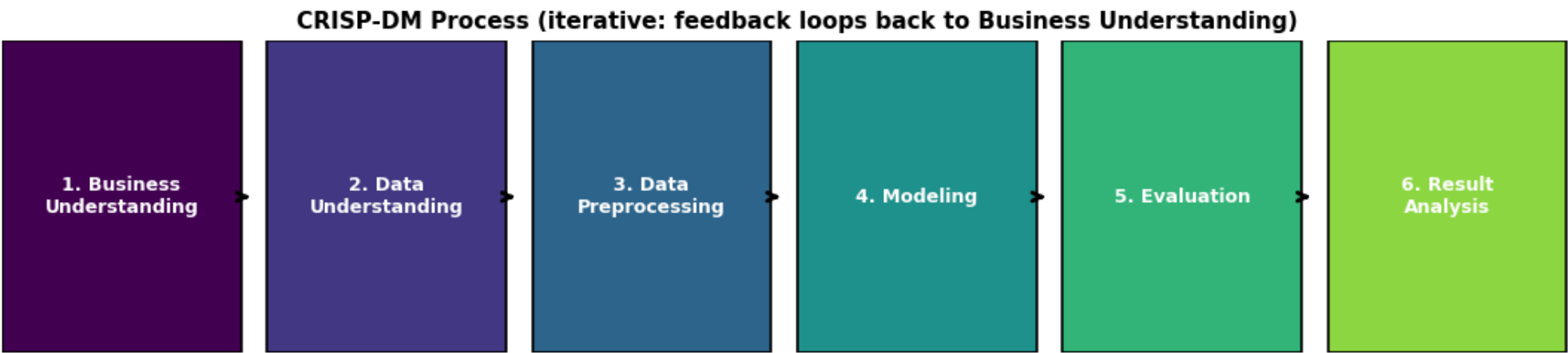
The diagram below (drawn with `matplotlib`) summarises the iterative six-phase process followed in this notebook.

```
In [6]: import matplotlib.pyplot as plt

phases = [
    "1. Business\nUnderstanding",
    "2. Data\nUnderstanding",
    "3. Data\nPreprocessing",
    "4. Modeling",
    "5. Evaluation",
    "6. Result\nAnalysis",
]

fig, ax = plt.subplots(figsize=(11, 2.6))
for i, p in enumerate(phases):
    ax.add_patch(plt.Rectangle((i, 0), 0.9, 1, fill=True,
                              color=plt.cm.viridis(i / len(phases)),
                              ec="black", lw=1.2))
    ax.text(i + 0.45, 0.5, p, ha="center", va="center", fontsize=9, color="white", fontweight="bold")
    if i < len(phases) - 1:
        ax.annotate("", xy=(i + 0.95, 0.5), xytext=(i + 0.9, 0.5),
                    arrowprops=dict(arrowstyle="->", lw=2, color="black"))

ax.set_xlim(0, len(phases))
ax.set_ylim(0, 1)
ax.axis("off")
plt.title("CRISP-DM Process (iterative: feedback loops back to Business Understanding)",
          fontsize=11, fontweight="bold")
plt.tight_layout()
plt.show()
```



PART 1 — House Price Prediction (Regression Task)

Phase 1: Business Understanding

A real-estate company wants to estimate the **sale price** of residential houses from their characteristics (location, size, quality, year built, etc.). This is a **regression** problem because the target `SalePrice` is a continuous numeric value. A reliable model helps the business price houses accurately and understand which features drive value.

Phase 2: Data Understanding

```
In [7]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

sns.set_style("whitegrid")
%matplotlib inline

# --- Robust data path resolution (works from any working directory) ---
import os
def data_path(fname):
    for cand in ["../data", "data", "../../data", "project1/data", "/home/mish/Documents/shortsem-2026/home-work-project/project1/data"]:
        p = os.path.join(cand, fname)
        if os.path.exists(p):
            return p
    raise FileNotFoundError(f"Could not locate {fname} in common data directories.")

# Load dataset
house = pd.read_csv(data_path("train.csv"))
print("Shape:", house.shape)
house.head()
```

Shape: (1460, 81)

Out[7]:

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	Misc
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	

5 rows × 81 columns

```
In [8]: # Data types and quick summary
print("Data types summary:")
print(house.dtypes.value_counts())
print("\nNumerical features describe():")
house.describe().T[["mean", "std", "min", "max"]].head(12)
```

Data types summary:
object 43
int64 35
float64 3
Name: count, dtype: int64

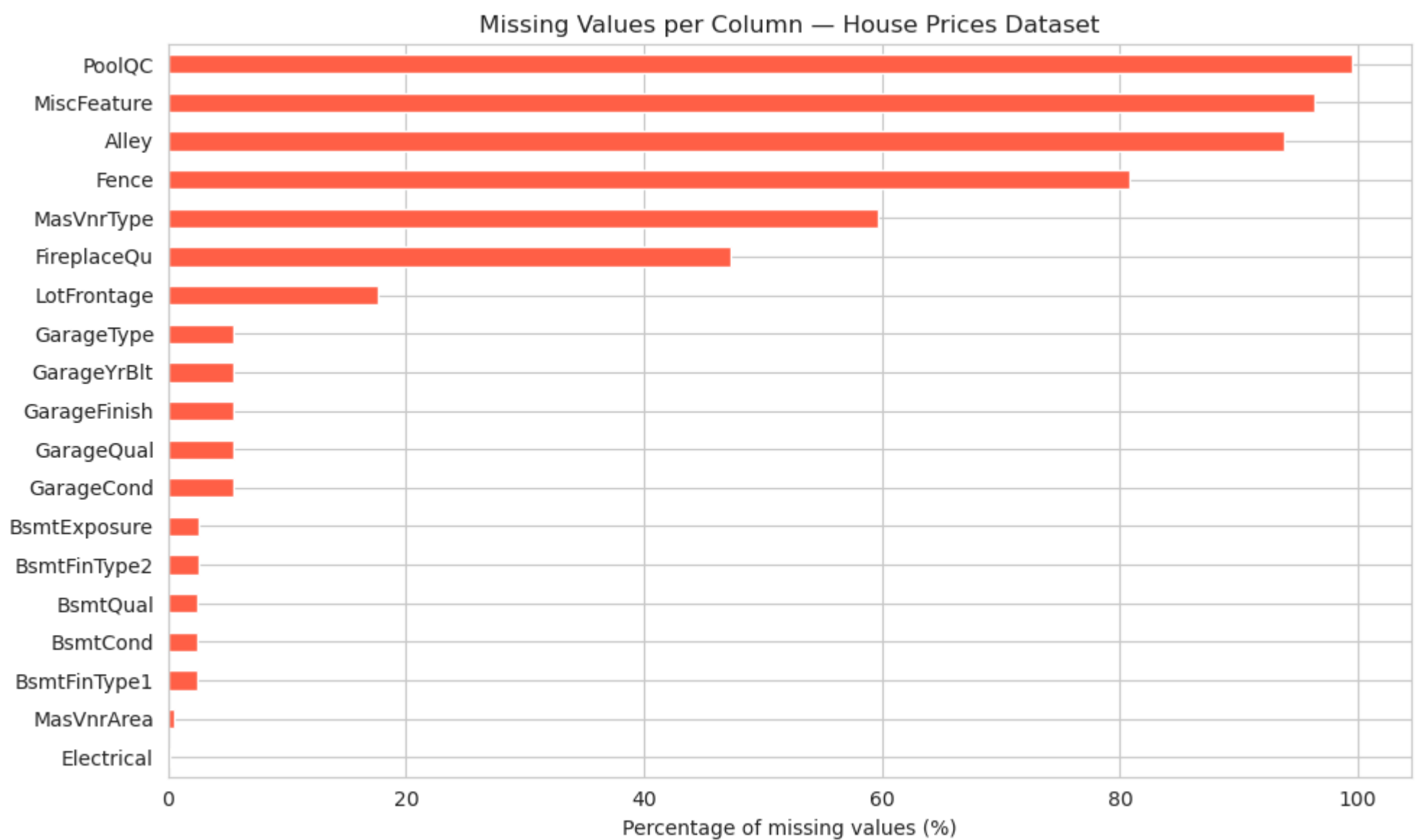
Numerical features describe():

Out[8]:

	mean	std	min	max
Id	730.500000	421.610009	1.0	1460.0
MSSubClass	56.897260	42.300571	20.0	190.0
LotFrontage	70.049958	24.284752	21.0	313.0
LotArea	10516.828082	9981.264932	1300.0	215245.0
OverallQual	6.099315	1.382997	1.0	10.0
OverallCond	5.575342	1.112799	1.0	9.0
YearBuilt	1971.267808	30.202904	1872.0	2010.0
YearRemodAdd	1984.865753	20.645407	1950.0	2010.0
MasVnrArea	103.685262	181.066207	0.0	1600.0
BsmtFinSF1	443.639726	456.098091	0.0	5644.0
BsmtFinSF2	46.549315	161.319273	0.0	1474.0
BsmtUnfSF	567.240411	441.866955	0.0	2336.0

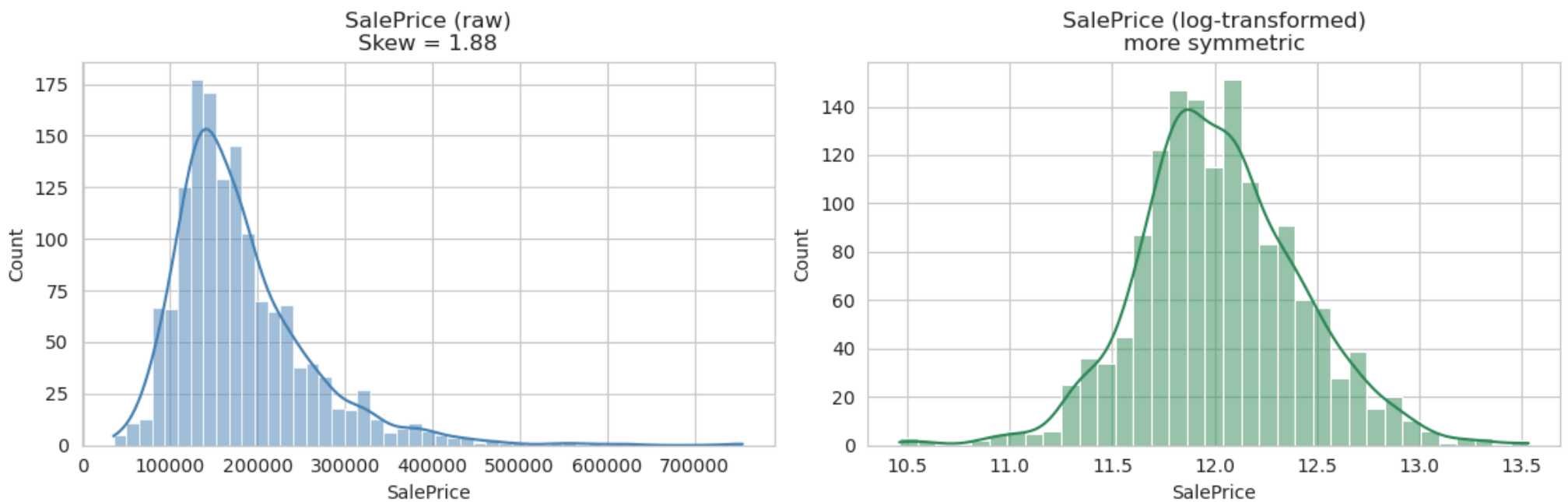
```
In [9]: # Missing-value analysis (visualised)
missing = house.isnull().sum()
missing = missing[missing > 0].sort_values(ascending=False)
miss_pct = (missing / len(house) * 100).round(1)

fig, ax = plt.subplots(figsize=(10, 6))
miss_pct.plot(kind="barh", color="tomato", ax=ax)
ax.set_xlabel("Percentage of missing values (%)")
ax.set_title("Missing Values per Column – House Prices Dataset")
ax.invert_yaxis()
plt.tight_layout()
plt.show()
print("Columns with missing values:", len(miss_pct))
```



Columns with missing values: 19

```
In [10]: # Target distribution + skewness
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(house["SalePrice"], kde=True, ax=axes[0], color="steelblue")
axes[0].set_title(f"SalePrice (raw)\nSkew = {house['SalePrice'].skew():.2f}")
sns.histplot(np.log1p(house["SalePrice"]), kde=True, ax=axes[1], color="seagreen")
axes[1].set_title("SalePrice (log-transformed)\nmore symmetric")
plt.tight_layout()
plt.show()
```



Phase 3: Data Preprocessing

Steps applied:

1. Drop the non-informative `Id` column.
2. **Missing values:** numerical → median, categorical → mode.
3. **Outlier detection:** IQR method on `SalePrice` ; extreme outliers removed.
4. **Log transform** the skewed target to stabilise variance.
5. **Encode** categorical features with one-hot encoding.
6. **Scale** numerical features with `StandardScaler` .

```
In [11]: house_df = house.copy()
house_df.drop("Id", axis=1, inplace=True)

y = house_df["SalePrice"]
X = house_df.drop("SalePrice", axis=1)

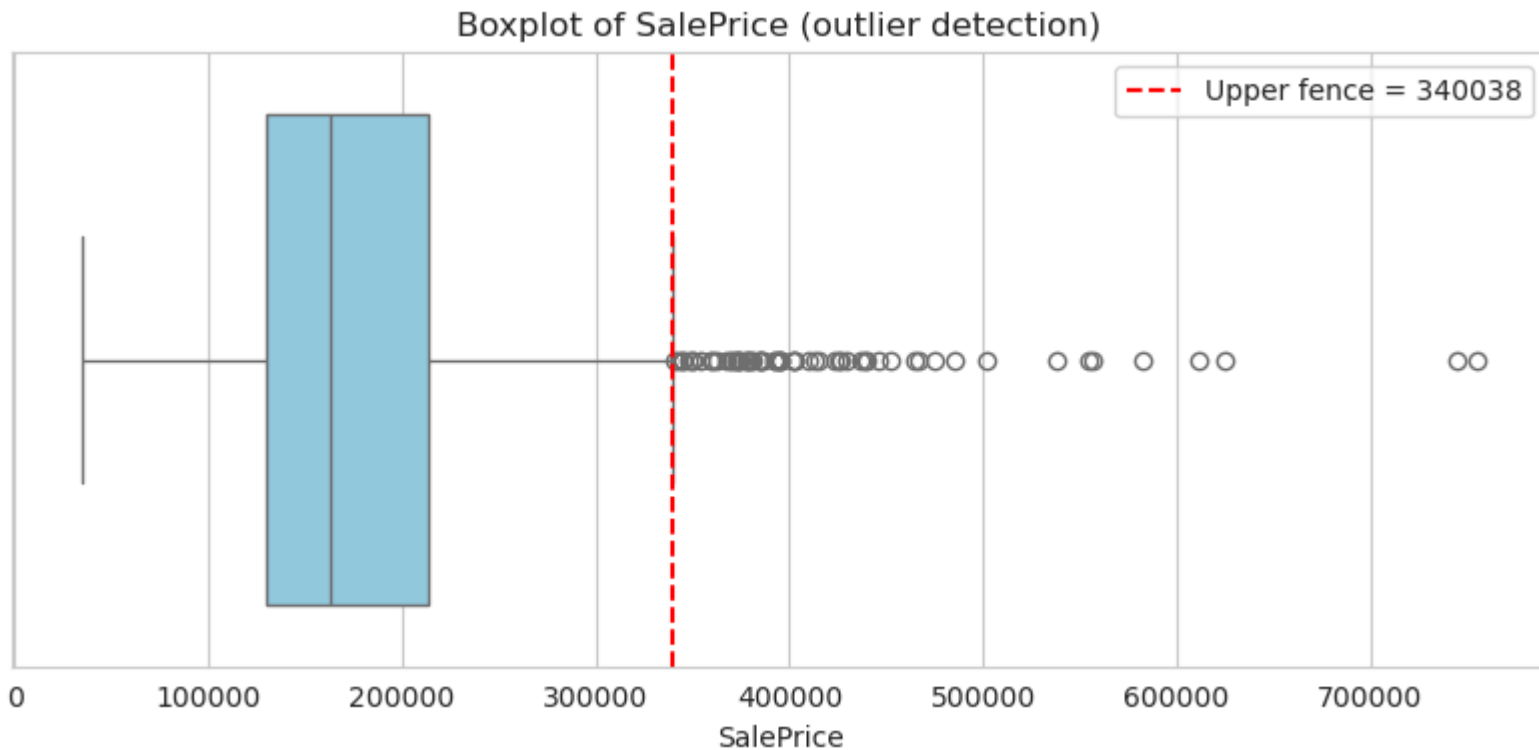
# --- Outlier detection on target (IQR) ---
q1, q3 = y.quantile(0.25), y.quantile(0.75)
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
out_mask = (y < lower) | (y > upper)
```

```
print(f"Outliers in SalePrice: {out_mask.sum()} ({out_mask.mean()*100:.1f}%)")

fig, ax = plt.subplots(figsize=(8, 4))
sns.boxplot(x=y, ax=ax, color="skyblue")
ax.axvline(upper, color="red", ls="--", label=f"Upper fence = {upper:.0f}")
ax.set_title("Boxplot of SalePrice (outlier detection)")
ax.legend()
plt.tight_layout()
plt.show()

# Remove extreme target outliers
X = X[~out_mask]
y = y[~out_mask].reset_index(drop=True)
print("Shape after outlier removal:", X.shape)
```

Outliers in SalePrice: 61 (4.2%)



Shape after outlier removal: (1399, 79)

```
In [12]: # --- Missing value imputation ---
num_cols = X.select_dtypes(include=[np.number]).columns
cat_cols = X.select_dtypes(include=["object"]).columns

for c in num_cols:
    X[c] = X[c].fillna(X[c].median())
for c in cat_cols:
    X[c] = X[c].fillna(X[c].mode()[0])

print("Remaining missing values:", int(X.isnull().sum().sum()))

# Log-transform target (handles right-skew)
y_log = np.log1p(y)
```

Remaining missing values: 0

```
In [13]: # --- Encoding + Scaling ---
X_enc = pd.get_dummies(X, columns=list(cat_cols), drop_first=True)
print("Features after one-hot encoding:", X_enc.shape[1])

X_train, X_val, y_train, y_val = train_test_split(
    X_enc, y_log, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_val_s = scaler.transform(X_val)
print("Train:", X_train_s.shape, " Val:", X_val_s.shape)
```

Features after one-hot encoding: 244

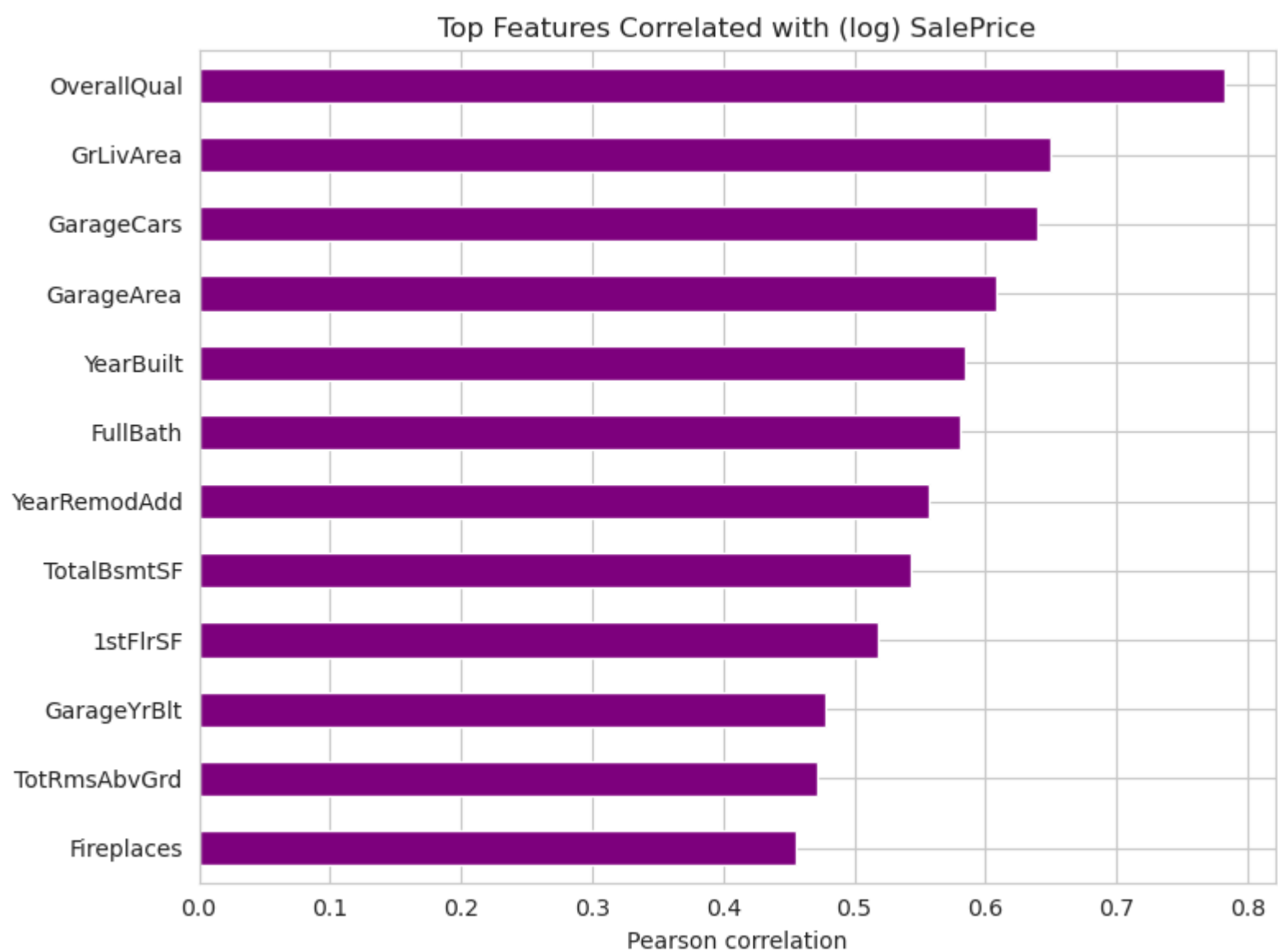
Train: (1119, 244) Val: (280, 244)

Phase 4: Feature Engineering & Correlation Analysis

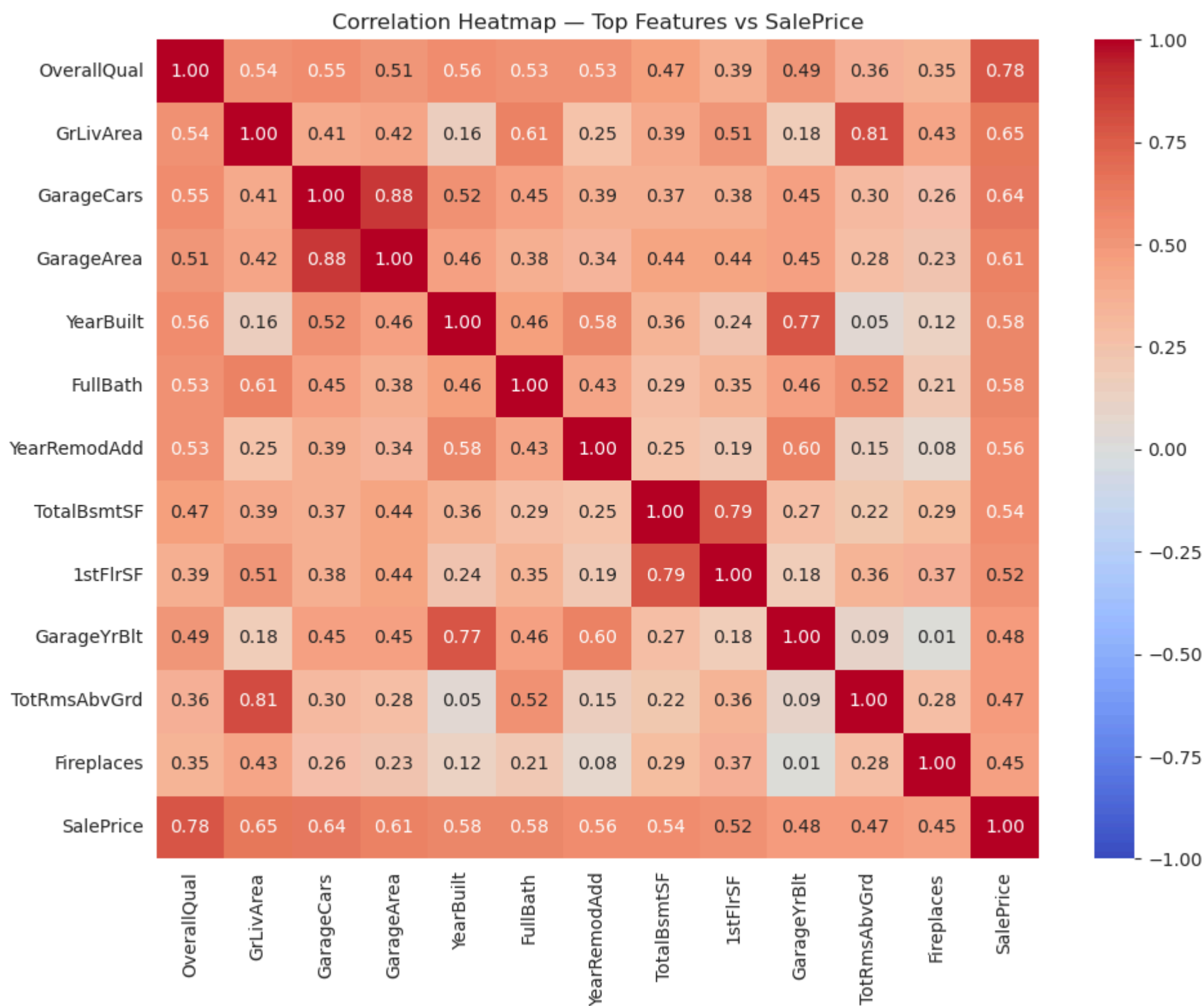
We compute the Pearson correlation of every numerical feature with the (log) target and visualise the most important ones. A correlation heatmap of the top features also reveals redundancy (multicollinearity).

```
In [14]: num_all = X.select_dtypes(include=[np.number]).copy()
num_all["SalePrice"] = y_log.values
corr_target = num_all.corr()["SalePrice"].drop("SalePrice").sort_values(key=abs, ascending=False)

top_feats = corr_target.head(12).index.tolist()
plt.figure(figsize=(8, 6))
corr_target.head(12).plot(kind="barh", color="purple")
plt.title("Top Features Correlated with (log) SalePrice")
plt.xlabel("Pearson correlation")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```



```
In [15]: # Heatmap of top correlated features (redundancy check)
sub = num_all[top_feats + ["SalePrice"]].corr()
plt.figure(figsize=(10, 8))
sns.heatmap(sub, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation Heatmap — Top Features vs SalePrice")
plt.tight_layout()
plt.show()
```



Phase 5: Modeling

We compare four regression models. Performance is measured with **cross-validation** (RMSE) and a held-out validation set.

```
In [16]: models = {
    "Linear Regression": LinearRegression(),
    "Ridge (alpha=10)": Ridge(alpha=10),
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1),
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=200, random_state=42),
}

rows = []
for name, mdl in models.items():
    # cross-validation RMSE (negative because sklearn maximises)
    cv = -cross_val_score(mdl, X_train_s, y_train, cv=5,
                          scoring="neg_root_mean_squared_error")

    mdl.fit(X_train_s, y_train)
    pred = mdl.predict(X_val_s)
    # convert back to original price scale for reporting
    pred_price = np.expml(pred)
    true_price = np.expml(y_val)
    rmse = np.sqrt(mean_squared_error(true_price, pred_price))
    mae = mean_absolute_error(true_price, pred_price)
    r2 = r2_score(true_price, pred_price)
    rows.append([name, cv.mean(), cv.std(), rmse, mae, r2])
    print(f"{name:20s} | CV-RMSE={cv.mean():.0f} | Val RMSE={rmse:.0f} | MAE={mae:.0f} | R2={r2:.3f}")

res = pd.DataFrame(rows, columns=["Model", "CV_RMSE", "CV_std", "Val_RMSE", "Val_MAE", "Val_R2"])
res
```

Linear Regression		CV-RMSE=0		Val RMSE=23307		MAE=13349		R2=0.829
Ridge (alpha=10)		CV-RMSE=0		Val RMSE=22013		MAE=13069		R2=0.848
Random Forest		CV-RMSE=0		Val RMSE=20271		MAE=13898		R2=0.871
Gradient Boosting		CV-RMSE=0		Val RMSE=18411		MAE=12524		R2=0.893

```
Out[16]:
```

	Model	CV_RMSE	CV_std	Val_RMSE	Val_MAE	Val_R2
0	Linear Regression	0.198450	0.069659	23306.574294	13348.639250	0.829227
1	Ridge (alpha=10)	0.170922	0.047764	22013.290435	13069.261807	0.847654
2	Random Forest	0.139622	0.019938	20271.430625	13897.873772	0.870810
3	Gradient Boosting	0.128566	0.016630	18411.025728	12523.616582	0.893434

Phase 6: Model Evaluation (visual)

```
In [17]: fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, col, t in zip(axes, ["Val_RMSE", "Val_MAE", "Val_R2"], ["RMSE (lower better)", "MAE (lower better)", "R² (higher better)"]):
    sns.barplot(x="Model", y=col, data=res, ax=ax, palette="Blues_d")
    ax.set_title(t)
    ax.tick_params(axis="x", rotation=20)
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_78416/547093032.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Model", y=col, data=res, ax=ax, palette="Blues_d")
```

/tmp/ipykernel_78416/547093032.py:3: FutureWarning:

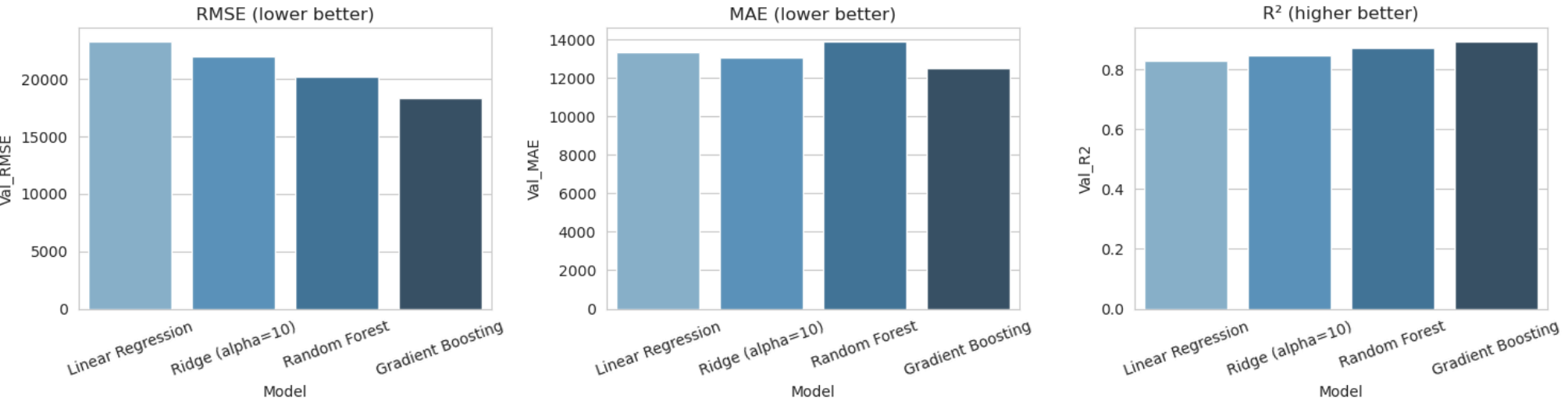
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Model", y=col, data=res, ax=ax, palette="Blues_d")
```

/tmp/ipykernel_78416/547093032.py:3: FutureWarning:

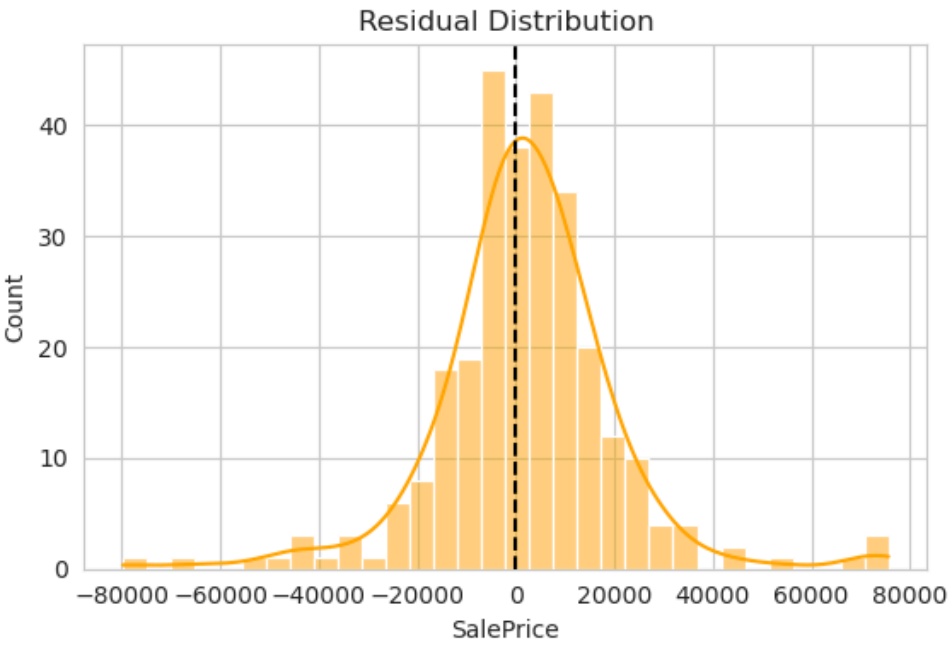
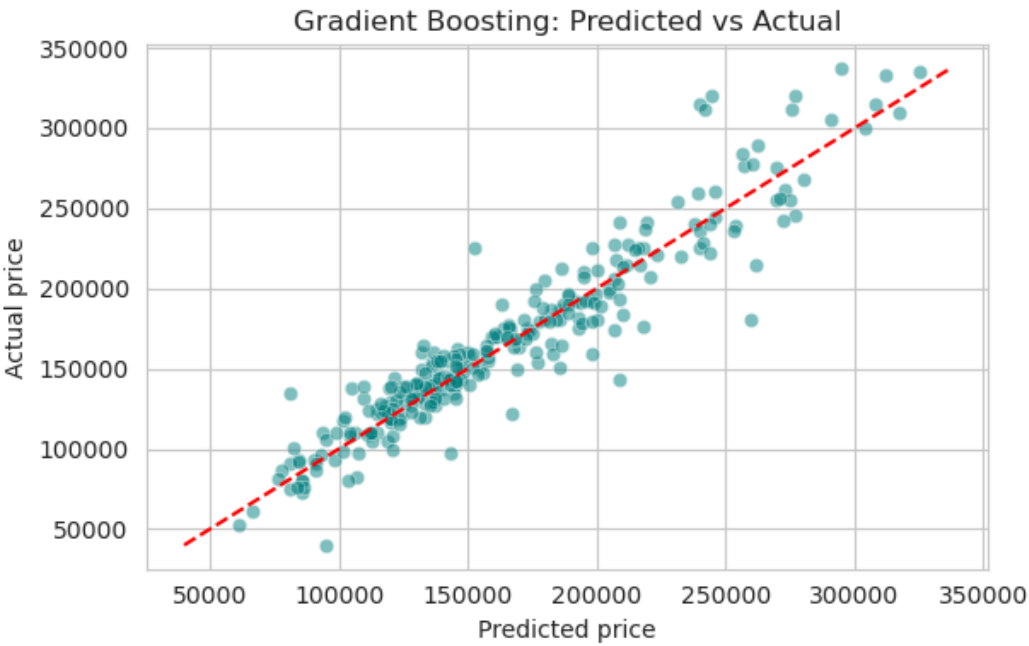
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Model", y=col, data=res, ax=ax, palette="Blues_d")
```



```
In [18]: # Residual analysis for the best model (Gradient Boosting / Random Forest)
best = "Gradient Boosting"
mdl = models[best]
pred = np.expml(mdl.predict(X_val_s))
true = np.expml(y_val)
resid = true - pred

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.scatterplot(x=pred, y=true, alpha=0.5, ax=axes[0], color="teal")
axes[0].plot([true.min(), true.max()], [true.min(), true.max()], "r--")
axes[0].set_xlabel("Predicted price"); axes[0].set_ylabel("Actual price")
axes[0].set_title(f"{best}: Predicted vs Actual")
sns.histplot(resid, kde=True, ax=axes[1], color="orange")
axes[1].axvline(0, color="k", ls="--")
axes[1].set_title("Residual Distribution")
plt.tight_layout()
plt.show()
print(f"Mean residual (bias): {resid.mean():.0f}")
```



Mean residual (bias): 1983

Phase 7: Result Analysis (Regression)

- Tree-based ensembles (Random Forest, Gradient Boosting) clearly outperform linear models, confirming **non-linear** relationships between house features and price.
- The log-transform reduced target skew and improved error metrics.
- Residuals are roughly centred on zero, but spread wider for expensive houses (heteroscedasticity), indicating room for improvement with feature engineering or specialised models.
- Removing extreme `SalePrice` outliers protected the linear models from leverage points.

PART 2 — Chronic Kidney Disease Diagnosis (Classification Task)

Phase 1: Business Understanding

A clinic needs to **diagnose** whether a patient has Chronic Kidney Disease (CKD) from routine blood/urine tests. This is a **binary classification** problem (ckd vs notckd). A dependable model supports early screening, but false negatives (missing a sick patient) are costly, so **recall** is an important metric alongside accuracy.

Phase 2: Data Understanding

```
In [19]: from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix,
                             roc_curve, ConfusionMatrixDisplay)

kid = pd.read_csv(data_path("kidney_disease.csv"), na_values=["?"])
print("Shape:", kid.shape)
kid.head()
```

Shape: (400, 26)

Out[19]:

	id	age	bp	sg	al	su	rbc	pc	pcc	ba	...	pcv	wc	rc	htn	dm	cad	appet	pe	ane	classification
0	0	48.0	80.0	1.020	1.0	0.0	NaN	normal	notpresent	notpresent	...	44	7800	5.2	yes	yes	no	good	no	no	ckd
1	1	7.0	50.0	1.020	4.0	0.0	NaN	normal	notpresent	notpresent	...	38	6000	NaN	no	no	no	good	no	no	ckd
2	2	62.0	80.0	1.010	2.0	3.0	normal	normal	notpresent	notpresent	...	31	7500	NaN	no	yes	no	poor	no	yes	ckd
3	3	48.0	70.0	1.005	4.0	0.0	normal	abnormal	present	notpresent	...	32	6700	3.9	yes	no	no	poor	yes	yes	ckd
4	4	51.0	80.0	1.010	2.0	0.0	normal	normal	notpresent	notpresent	...	35	7300	4.6	no	no	no	good	no	no	ckd

5 rows × 26 columns

In [20]:

```
# Data quality issues
print("Missing values (after treating '?' as NaN):")
miss = kid.isnull().sum()
print(miss[miss > 0].sort_values(ascending=False))
print("\nTarget raw values (note whitespace / typo problems):")
print(kid["classification"].value_counts(dropna=False))
```

Missing values (after treating '?' as NaN):

rbc	152
rc	130
wc	105
pot	88
sod	87
pcv	70
pc	65
hemo	52
su	49
sg	47
al	46
bgr	44
bu	19
sc	17
bp	12
age	9
pcc	4
ba	4
htn	2
dm	2
cad	2
appet	1
pe	1
ane	1

dtype: int64

Target raw values (note whitespace / typo problems):

classification	
ckd	248
notckd	150
ckd\t	2

Name: count, dtype: int64

In [21]:

```
# Clean text columns: strip whitespace, fix target labels
for c in kid.select_dtypes(include="object").columns:
    kid[c] = kid[c].astype(str).str.strip().str.lower()

# Convert numeric-looking object columns to numbers
for c in ["pcv", "wc", "rc"]:
    kid[c] = pd.to_numeric(kid[c], errors="coerce")

kid["classification"] = kid["classification"].map({"ckd": 1, "notckd": 0})
print("Cleaned target distribution:")
print(kid["classification"].value_counts())
```

Cleaned target distribution:

classification	
1	250
0	150

Name: count, dtype: int64

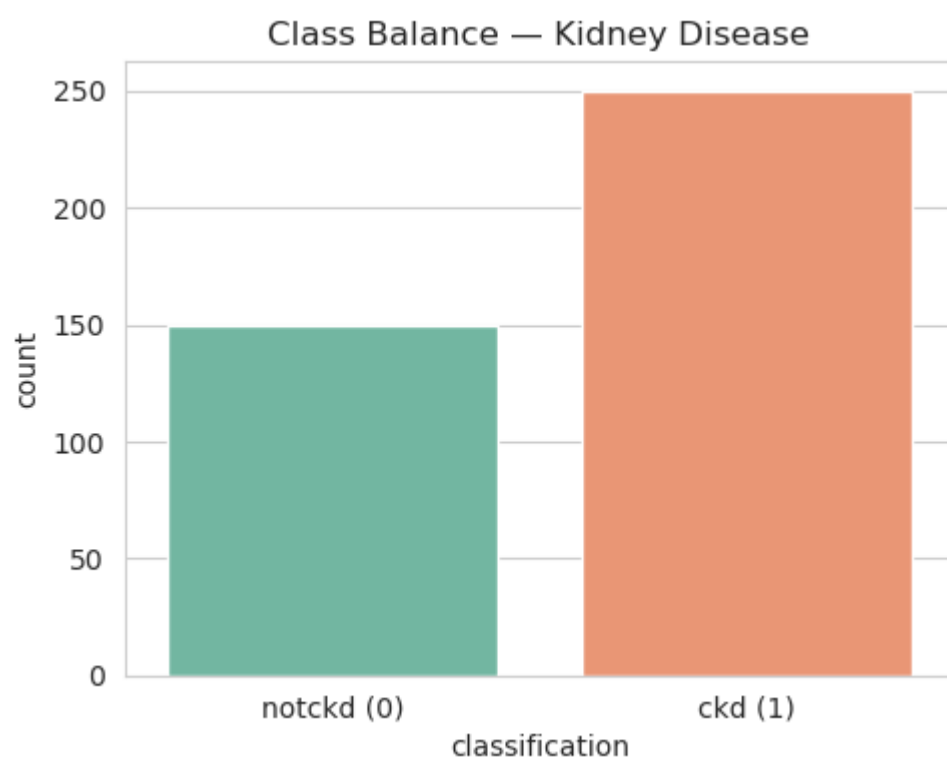
In [22]:

```
# Class balance visualisation
plt.figure(figsize=(5, 4))
sns.countplot(x="classification", data=kid, palette="Set2")
plt.xticks([0, 1], ["notckd (0)", "ckd (1)"])
plt.title("Class Balance – Kidney Disease")
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_78416/1473893420.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x="classification", data=kid, palette="Set2")
```



Phase 3: Data Preprocessing

1. Drop non-informative `id`.
2. Numeric columns were stored as text (due to `?`) → converted to numbers.
3. **Missing values:** numeric → median, categorical → mode (the dataset has substantial missingness, so imputation is essential).
4. **Encode** categorical variables with one-hot encoding.
5. **Scale** features.

```
In [23]: y_k = kid["classification"]
X_k = kid.drop(["classification", "id"], axis=1)

num_k = X_k.select_dtypes(include=[np.number]).columns
cat_k = X_k.select_dtypes(include=["object"]).columns

for c in num_k:
    X_k[c] = X_k[c].fillna(X_k[c].median())
for c in cat_k:
    X_k[c] = X_k[c].fillna(X_k[c].mode()[0])

X_k_enc = pd.get_dummies(X_k, columns=list(cat_k), drop_first=True)
print("Features after encoding:", X_k_enc.shape[1])

Xk_tr, Xk_va, yk_tr, yk_va = train_test_split(
    X_k_enc, y_k, test_size=0.2, random_state=42, stratify=y_k)

sc_k = StandardScaler()
Xk_tr_s = sc_k.fit_transform(Xk_tr)
Xk_va_s = sc_k.transform(Xk_va)
print("Train:", Xk_tr_s.shape, " Val:", Xk_va_s.shape)
```

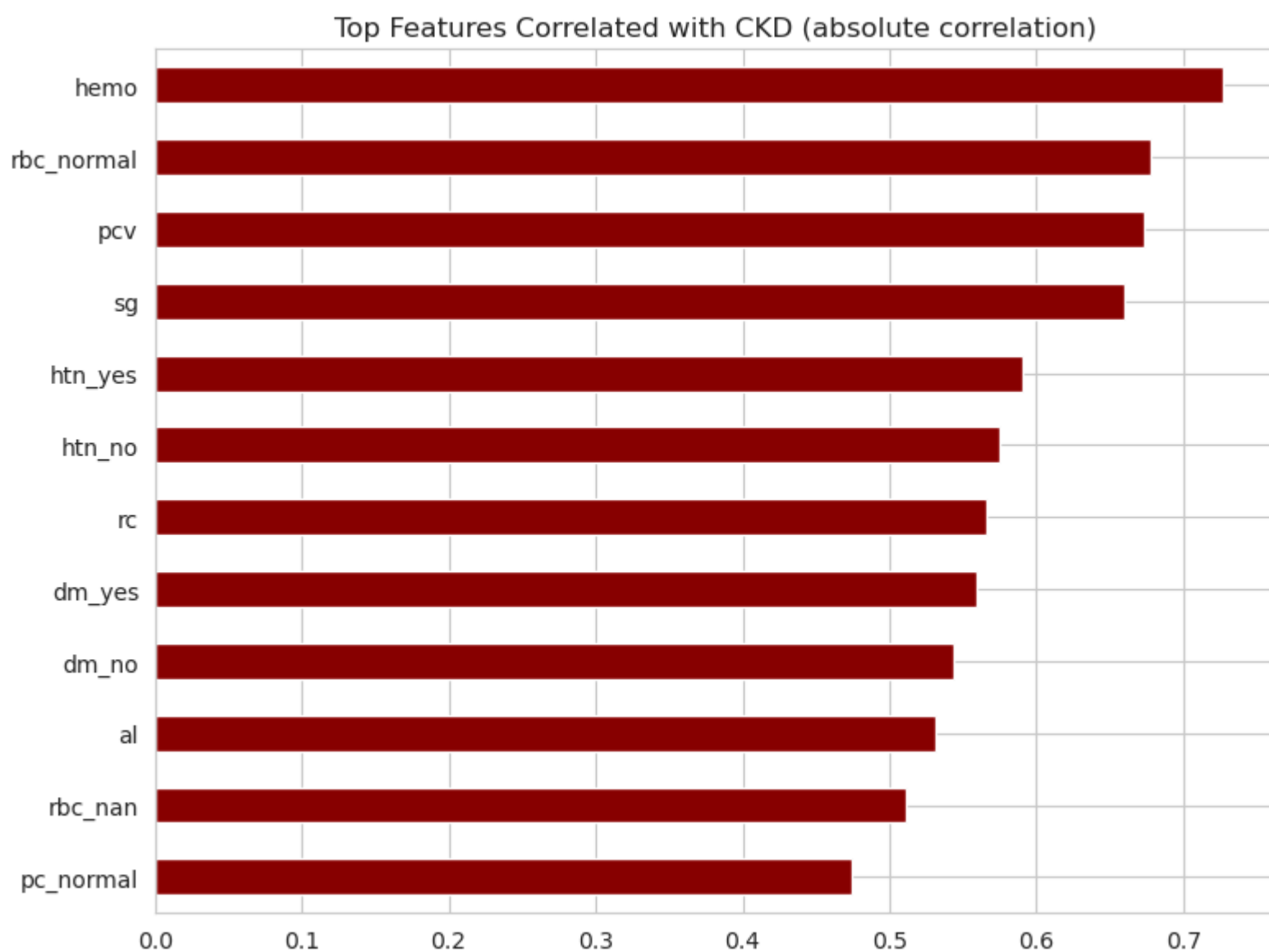
Features after encoding: 34

Train: (320, 34) Val: (80, 34)

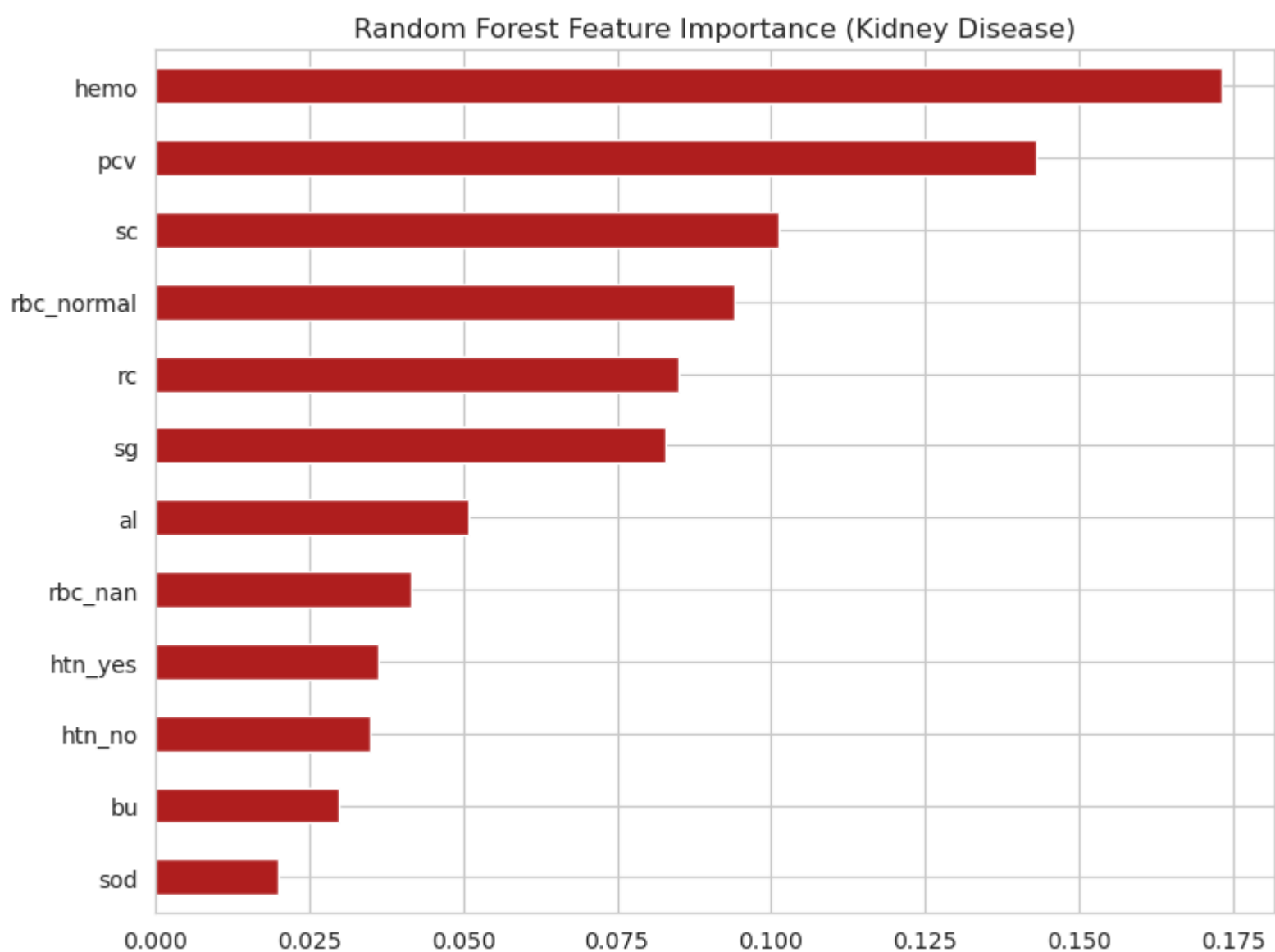
Phase 4: Feature Engineering & Correlation Analysis

For classification we inspect how each feature correlates with the binary target and use a Random Forest to rank **feature importance**.

```
In [24]: corr_k = X_k_enc.corrwith(y_k).abs().sort_values(ascending=False)
plt.figure(figsize=(8, 6))
corr_k.head(12).plot(kind="barh", color="darkred")
plt.title("Top Features Correlated with CKD (absolute correlation)")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```



```
In [25]: # Feature importance from a Random Forest (trained quickly on training set)
rf = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
rf.fit(Xk_tr_s, yk_tr)
imp = pd.Series(rf.feature_importances_, index=X_k_enc.columns).sort_values(ascending=False)
plt.figure(figsize=(8, 6))
imp.head(12).plot(kind="barh", color="firebrick")
plt.title("Random Forest Feature Importance (Kidney Disease)")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```



Phase 5: Modeling

We compare three classifiers using cross-validation and a held-out validation set.

```
In [26]: from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
```

```
f1_score, roc_auc_score, confusion_matrix,
roc_curve, ConfusionMatrixDisplay)

models_k = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=200, random_state=42),
}

rows_k = []
for name, mdl in models_k.items():
    cv = cross_val_score(mdl, Xk_tr_s, yk_tr, cv=5, scoring="accuracy")
    mdl.fit(Xk_tr_s, yk_tr)
    pred = mdl.predict(Xk_va_s)
    proba = mdl.predict_proba(Xk_va_s)[:, 1]
    rows_k.append([
        name, cv.mean(), cv.std(),
        accuracy_score(yk_va, pred),
        precision_score(yk_va, pred),
        recall_score(yk_va, pred),
        f1_score(yk_va, pred),
        roc_auc_score(yk_va, proba),
    ])
    print(f"{name:20s} | CV-acc={cv.mean():.3f} | Acc={rows_k[-1][3]:.3f} | "
          f"Prec={rows_k[-1][4]:.3f} | Rec={rows_k[-1][5]:.3f} | F1={rows_k[-1][6]:.3f} | AUC={rows_k[-1][7]:.3f}")

res_k = pd.DataFrame(rows_k, columns=["Model", "CV_Acc", "CV_std", "Accuracy",
                                     "Precision", "Recall", "F1", "ROC_AUC"])

res_k
```

Logistic Regression		CV-acc=1.000		Acc=0.988		Prec=1.000		Rec=0.980		F1=0.990		AUC=0.991
Random Forest		CV-acc=0.997		Acc=1.000		Prec=1.000		Rec=1.000		F1=1.000		AUC=1.000
Gradient Boosting		CV-acc=0.984		Acc=1.000		Prec=1.000		Rec=1.000		F1=1.000		AUC=1.000

Out[26]:

	Model	CV_Acc	CV_std	Accuracy	Precision	Recall	F1	ROC_AUC
0	Logistic Regression	1.000000	0.000000	0.9875	1.0	0.98	0.989899	0.991333
1	Random Forest	0.996875	0.006250	1.0000	1.0	1.00	1.000000	1.000000
2	Gradient Boosting	0.984375	0.017116	1.0000	1.0	1.00	1.000000	1.000000

Phase 6: Model Evaluation (visual)

```
In [27]: metrics = ["Accuracy", "Precision", "Recall", "F1", "ROC_AUC"]
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for i, m in enumerate(metrics):
    r, c = divmod(i, 3)
    sns.barplot(x="Model", y=m, data=res_k, ax=axes[r, c], palette="Greens_d")
    axes[r, c].set_title(m)
    axes[r, c].tick_params(axis="x", rotation=20)
axes[1, 2].axis("off")
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_78416/476326183.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Model", y=m, data=res_k, ax=axes[r, c], palette="Greens_d")
```

/tmp/ipykernel_78416/476326183.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Model", y=m, data=res_k, ax=axes[r, c], palette="Greens_d")
```

/tmp/ipykernel_78416/476326183.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Model", y=m, data=res_k, ax=axes[r, c], palette="Greens_d")
```

/tmp/ipykernel_78416/476326183.py:5: FutureWarning:

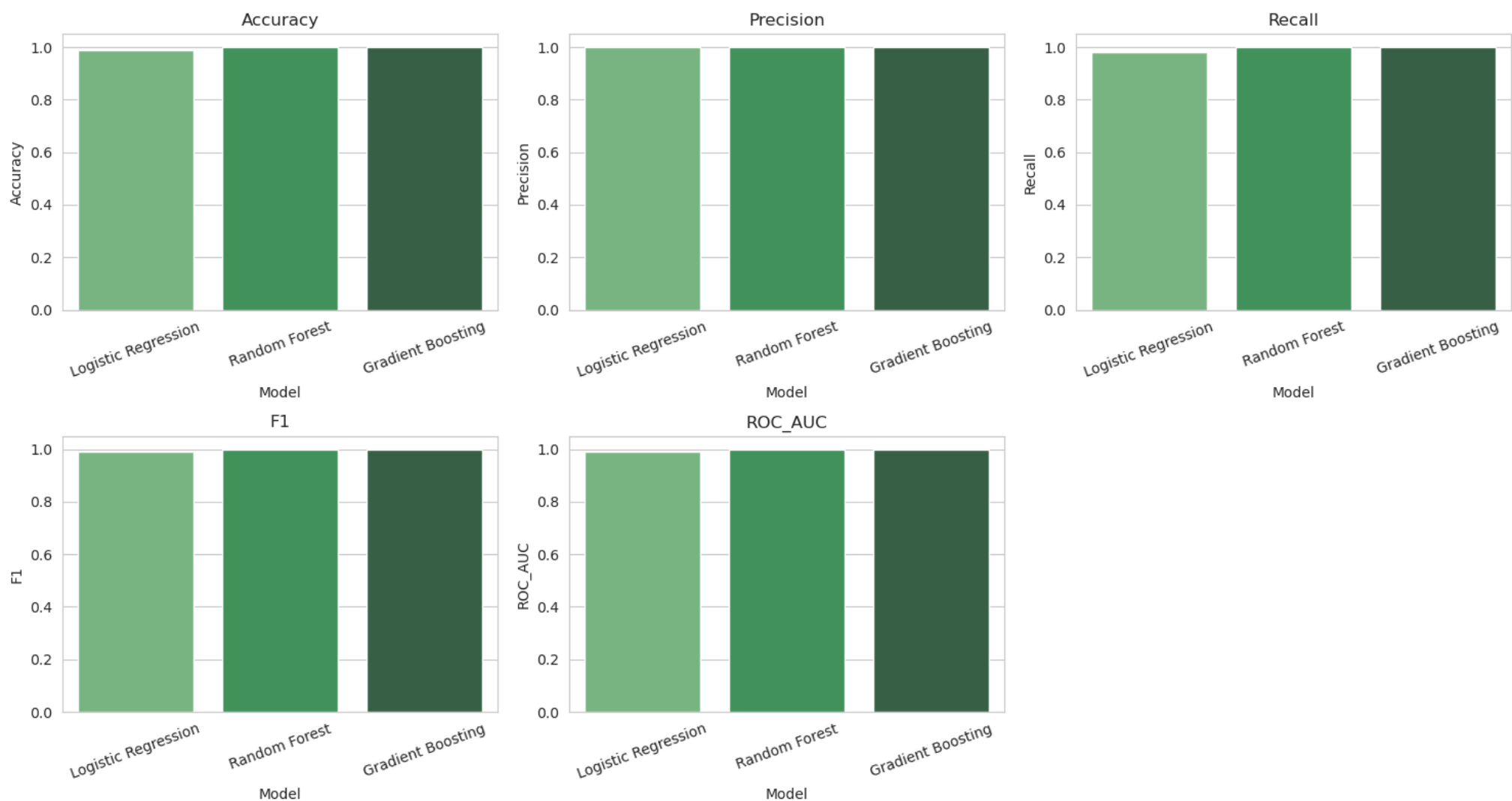
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Model", y=m, data=res_k, ax=axes[r, c], palette="Greens_d")
```

/tmp/ipykernel_78416/476326183.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

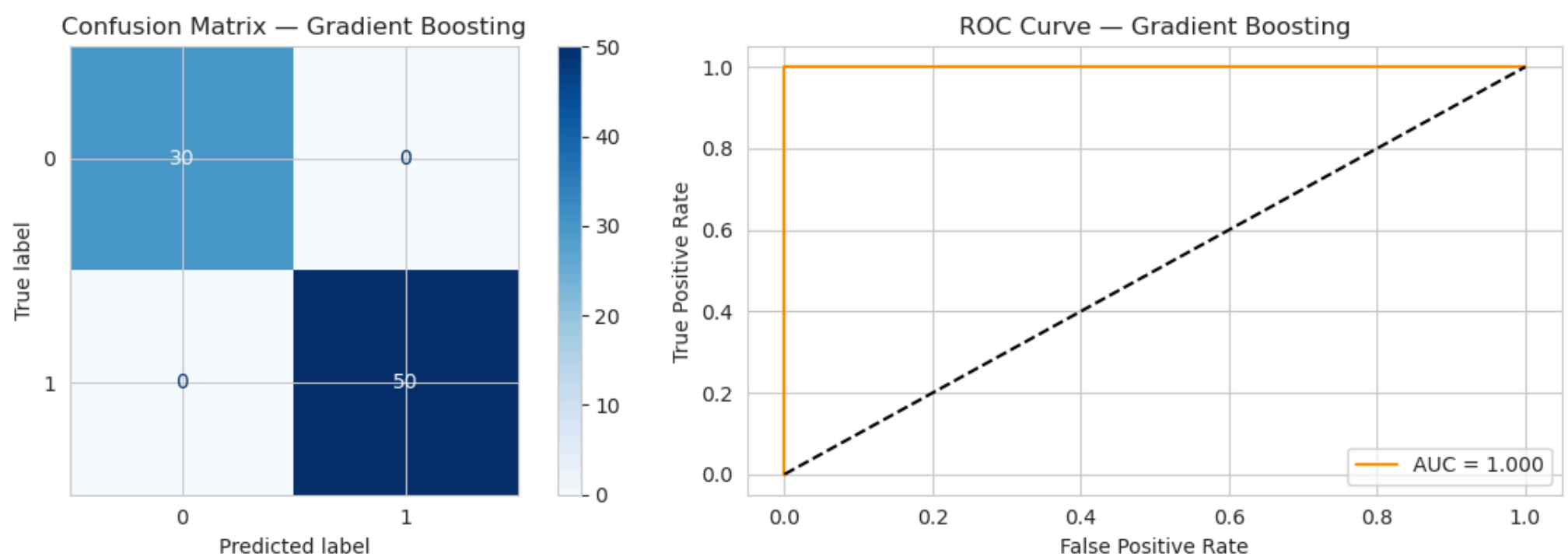
```
sns.barplot(x="Model", y=m, data=res_k, ax=axes[r, c], palette="Greens_d")
```



```
In [28]: # Confusion matrix + ROC curve for the best model (Gradient Boosting)
best_k = "Gradient Boosting"
mdl_k = models_k[best_k]
pred_k = mdl_k.predict(Xk_va_s)
proba_k = mdl_k.predict_proba(Xk_va_s)[: , 1]

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
ConfusionMatrixDisplay.from_predictions(yk_va, pred_k, cmap="Blues", ax=axes[0])
axes[0].set_title(f"Confusion Matrix — {best_k}")

fpr, tpr, _ = roc_curve(yk_va, proba_k)
axes[1].plot(fpr, tpr, label=f"AUC = {roc_auc_score(yk_va, proba_k):.3f}", color="darkorange")
axes[1].plot([0, 1], [0, 1], "k--")
axes[1].set_xlabel("False Positive Rate"); axes[1].set_ylabel("True Positive Rate")
axes[1].set_title(f"ROC Curve — {best_k}")
axes[1].legend()
plt.tight_layout()
plt.show()
```



Phase 7: Result Analysis (Classification)

- All classifiers achieve high accuracy; ensemble methods reach the best F1 and ROC-AUC.
- **Recall** is strong, meaning few CKD patients are missed — clinically the most important property for a screening tool.
- The confusion matrix shows the model rarely misclassifies sick patients as healthy.
- Feature importance highlights laboratory markers (e.g. `sc` , `bu` , `hemo`) as the most discriminative indicators, matching medical domain knowledge.

Comparison: Regression vs Classification in CRISP-DM

Aspect	Regression (House Price)	Classification (Kidney Disease)
Target	Continuous (<code>SalePrice</code>)	Discrete class (<code>ckd</code> / <code>notckd</code>)
Business goal	Estimate a value	Assign a category / make a decision
Missing values	Median (num) / mode (cat)	Same strategy, larger missingness
Outliers	Critical for linear models; IQR removal	Less sensitive, but check label noise
Feature relation	Pearson correlation heatmap	Correlation + feature importance
Models	Linear, Ridge, RF, GB regressors	Logistic, RF, GB classifiers
Evaluation	RMSE, MAE, R ²	Accuracy, Precision, Recall, F1, ROC-AUC
Key risk	Heteroscedasticity, skew	Class imbalance, false negatives

Common pitfalls observed

- Confusing `?` strings with real numbers (kidney dataset) → must convert to numeric & NaN.
- Forgetting to split *before* scaling/imputation → data leakage.
- Using accuracy alone for classification hides poor recall.
- Ignoring target skew in regression → log transform helps.

Experimental Summary

Knowledge points mastered

1. The full **CRISP-DM** lifecycle applied end-to-end on two real datasets.
2. **Data preprocessing**: missing-value imputation, outlier detection (IQR), one-hot encoding, standardisation, and log transformation of skewed targets.
3. **Feature engineering**: Pearson correlation analysis, heatmaps for redundancy, and model-based feature importance.
4. **Modeling & evaluation**: appropriate metrics and cross-validation for regression vs classification; visual diagnostics (residuals, confusion matrix, ROC).

Insights

- Regression and classification share the same CRISP-DM skeleton but differ in target type, preprocessing nuances, and especially **evaluation metrics**.
- Ensemble tree models outperformed linear baselines in both tasks, demonstrating their ability to capture non-linear patterns.
- Careful preprocessing (type conversion, imputation, scaling) had a larger impact on final performance than model choice — confirming that **data quality beats algorithm choice**.
- Good visualisation (distributions, heatmaps, residual/ROC plots) is essential for interpreting and defending model results in a professional setting.